# Guião · Margarida

Experiências no AnyLogic (instrumentação, cenários, otimização), ESMA e investidor alavancado

SAD · Grupo 2 · Volatility Interruptions and Crash Contagion in Euronext Lisbon · entrega a 2 de dezembro. Os nomes vêm de `docs/CONTRATO.md`.

## Resumo das tuas semanas

| Semana | Entrega concreta |
|---|---|
| 1 (17–24 out) | Aprender as experiências do AnyLogic (Parameters Variation, Optimization); notas da ESMA. |
| 2 (24–31 out) | **Instrumentar o modelo**: parâmetros de cenário, ficheiros CSV de saída e KPIs (secção 2). |
| 3 (31 out–7 nov) | Montar as experiências dos cenários e fazer as primeiras corridas (o João testa o `kpis.py` com elas). |
| 4 (7–14 nov) | **Calibração** com o João: tu corres o baseline e mudas os parâmetros, ele compara com o BCP. |
| 5 (14–21 nov) | Correr os cenários da Fase 1 com 30 replicações. Fase 2: **LeveragedTrader**. |
| 6 (21–28 nov) | Corridas finais (grelha e otimização). Relatório: instrumentação, cenários e experiências, e o investidor alavancado. |

# 0. Como trabalhar (lê isto primeiro)

### Uma vez só

1. Aceita o convite do GitHub que chega ao teu email (repositório **sad-euronext-g2**).
2. Instala o **GitHub Desktop** (desktop.github.com) e entra com a tua conta.
3. No GitHub Desktop: **File** → **Clone repository** → **sad-euronext-g2** → **Clone**. Fica uma pasta no teu PC (por defeito em Documentos\GitHub\sad-euronext-g2).
4. Confirma com o grupo que **todos usam a mesma versão do AnyLogic** (Help → About). Um modelo gravado numa versão mais recente pode não abrir numa mais antiga.

### Todos os dias

1. GitHub Desktop: **Fetch origin** → **Pull** (trazes o que os outros fizeram).
2. AnyLogic: **File** → **Open** e abres o teu `.alp` dentro da pasta do repositório. Não reinstalas nada: o GitHub só mantém a pasta sincronizada.
3. Trabalhas e gravas (Ctrl+S).
4. GitHub Desktop: escreves uma mensagem curta (ex.: `João: fundamentalista com statechart`) → **Commit to main** → **Push origin**.

### Onde trabalhas

- `model/Euronext_Main.alp` é o modelo principal e **só o Miguel o edita**.
- Tu trabalhas em `sandbox/margarida/`. Depois do Miguel publicar o modelo base, copia `model/Euronext_Main.alp` para a tua pasta com outro nome (ex.: `sandbox/margarida/margarida.alp`) e abre esse. Na tua cópia podes mexer em tudo, incluindo no Main.
- Quando a tua peça funcionar: commit + push e avisa no grupo. O Miguel abre os dois modelos no AnyLogic e, no painel **Projects**, copia o teu agente (Ctrl+C) e cola-o no modelo principal (Ctrl+V). Isto só funciona sem erros se usares os nomes do `docs/CONTRATO.md`.
- Se estragares a tua cópia: GitHub Desktop → separador **History** → botão direito num commit antigo → **Revert**. Ou voltas a copiar o modelo principal.

### AnyLogic em 1 minuto

| Coisa | Para que serve | Onde está |
|---|---|---|
| Agent type | um tipo de participante (ex.: `Fundamentalist`) | Palette → Agent → arrastar **Agent** |
| Population | muitos agentes do mesmo tipo dentro do Main | assistente do Agent → *Population of agents* |
| Parameter | valor fixo que muda de cenário para cenário (ex.: `nChartists`) | Palette → Agent → Parameter |
| Variable | valor que muda durante a simulação (ex.: `cash`) | Palette → Agent → Variable |
| Function | bloco de código que se chama pelo nome | Palette → Agent → Function |
| Event | código que corre num instante ou de X em X minutos | Palette → Agent → Event |
| Statechart | os estados de um agente e o que o faz mudar de estado | Palette → Statechart |
| Process Modeling Library | blocos de filas (Enter, Queue, Hold, Delay, Sink…) | Palette → Process Modeling Library |
| `main` | dentro de qualquer agente, `main` dá acesso ao Main (ex.: `main.stock.mid()`) | — |

### Como desenhar um statechart

1. Abre o agente: duplo clique no tipo (ex.: `Fundamentalist`) no painel **Projects**.
2. Palette → **Statechart** → arrasta **Statechart Entry Point**. O nome fica `statechart`; não o mudes (o Miguel chama `statechart` em todos os agentes).
3. Arrasta um **State**, encosta-lhe a ponta do Entry Point e dá-lhe o nome em **Name**.
4. Transição: na Palette faz **duplo clique em Transition** (fica em modo de desenho), clica na borda do estado de origem e depois na borda do estado de destino.
5. Clica na transição e escolhe **Triggered by**: **Rate** (acontece em média X vezes por minuto, ao acaso: "de vez em quando decide"); **Timeout** (X minutos depois de entrar no estado; com Timeout 0 o agente faz a ação e volta logo); **Message** (quando chega uma mensagem: Message type `String`, Fire transition *On particular message*, Message `"HALT"`); **Condition** (usada à saída de um Branch).
6. **Branch** (o losango na Palette Statechart) serve para decidir. Cada seta que sai dele tem uma Condition; uma tem de ser **Default** (a que se segue se nenhuma outra for verdadeira).
7. Código: cada estado tem **Entry action** (corre ao entrar) e cada transição tem **Action** (corre ao passar) e **Guard** (condição extra para a transição poder acontecer).
8. **Estado composto**: desenha um State grande e arrasta outros para dentro; dentro dele põe um **Initial State Pointer** apontado para o primeiro estado. Uma seta que sai do estado grande vale para todos os de dentro: é assim que se faz o `HALT` (de qualquer estado, o agente vai para `Halted`).
9. Testar: carrega em Run; na barra de cima da janela da simulação escolhe o agente (ex.: `fundamentalists[0]`) e vês o estado ativo a verde. Para escrever na consola: `traceln("texto " + x);`.

Este guião tem o código para colar. Onde aparece **Entry action**, **Action** ou o nome de uma função, copia o bloco cinzento para esse campo. Se o AnyLogic não aceitar `statechart.receiveMessage(msg)`, usa `statechart.fireEvent(msg)`, que faz o mesmo.

# 1. Semana 1: experiências do AnyLogic e ESMA

1. Lê o capítulo 0 deste guião.
2. No AnyLogic: Help → **AnyLogic Help** → lê as páginas “Parameter variation experiment” e “Optimization experiment”. Num modelo de teste qualquer, faz uma Parameters Variation com 1 parâmetro e 5 replicações, para ver como funciona. É isto que vais usar a partir da semana 3.
3. Lê a secção 3 deste guião (cenários) para saberes o que vais correr.

### ESMA (2020)

Procura “ESMA Market impacts of circuit breakers 2020” (é um ESMA Economic Report). Lê o resumo e os resultados e preenche esta tabela em `sandbox/margarida/esma.md`. São as metas qualitativas da validação:

| Efeito | O que a ESMA encontrou (sentido e número) | Página |
|---|---|---|
| volatilidade depois vs. antes da interrupção | | |
| spread depois vs. antes | | |
| volume depois vs. antes | | |
| preço do leilão vs. preço antes (continua a cair ou recupera?) | | |
| frequência de interrupções repetidas | | |

Dá a tabela ao João: ele usa-a na validação.

# 2. Semana 2: instrumentar o modelo (cenários, CSV e KPIs)

És tu que vais usar os dados que o modelo produz, por isso és tu que defines o que ele grava. Fazes isto no Main da tua cópia (`sandbox/margarida/margarida.alp`, copiada do modelo base do Miguel) e, no fim, o Miguel copia para o modelo principal. Também te serve de treino de AnyLogic para as experiências.

1. Cria a pasta `sandbox/margarida/output/`. Quando o Miguel integrar, a pasta passa a ser `model/output/`.
2. No Main, **parâmetros** (Palette → Agent → Parameter): `collarSet` int 2, `customDyn` 0.03, `customStat` 0.08, `scenario` String "baseline".
3. **Variáveis**: `label` String, `runId` String com Initial value `java.util.UUID.randomUUID().toString().substring(0, 8)`, `peakPrice` double, `maxDrawdown` double 0, `nHalts` int 0, `timeHalted` double 0.
4. Palette **Connectivity** → **Text File**: name `pricesFile`, File `output/precos.csv`, Mode **Append**. Outro igual: `haltsFile`, `output/halts.csv`.
5. Main → Properties → Agent actions → **On startup**:

```
switch (collarSet) {
    case 0: collarsActive = false; break;                                        // sem colares
    case 1: dynamicThreshold = 0.02; staticThreshold = 0.05; break;
    case 2: dynamicThreshold = 0.03; staticThreshold = 0.08; break;              // regras atuais
    case 3: dynamicThreshold = 0.05; staticThreshold = 0.10; break;
    case 4: dynamicThreshold = customDyn; staticThreshold = customStat; break; // otimização
}
peakPrice = initialPrice;
label = scenario + "_c" + collarSet + "_h" + haltShort;   // quando entrar o Chartist: + "_n" + nChartists
```

1. Event `logEvent` no Main: Mode *Cyclic*, First occurrence 1, Recurrence 1. Action:

```
double p = stock.lastPrice;
peakPrice = Math.max(peakPrice, p);
maxDrawdown = Math.max(maxDrawdown, 1 - p / peakPrice);
pricesFile.println(label + ";" + runId + ";" + time() + ";" + p + ";"
                   + stock.bestBid() + ";" + stock.bestAsk() + ";" + stock.volume);
```

1. Main → **On destroy**: `pricesFile.close(); haltsFile.close();`
2. No Stock, nas funções `startHalt` e `endHalt` do Miguel, descomenta as linhas com `main.nHalts`, `main.haltsFile`, `main.label`, `main.runId` e `main.timeHalted`.
3. **Teste:** corre a Simulation durante uns minutos e fecha. O `output/precos.csv` deve ter linhas como `baseline_c2_h3.0;a1b2c3d4;12.0;0.5013;0.501;0.5016;84200`. Com `noiseSigma = 0.01` devem aparecer linhas no `halts.csv`.
4. Commit e avisa o Miguel para integrar (ele tem os passos no guião dele, secção 6).

## 3. Correr o modelo e os cenários no AnyLogic (semanas 3–5)

### Onde saem os dados

Cada corrida acrescenta linhas a `model/output/precos.csv` (um preço por minuto) e `model/output/halts.csv` (uma linha por interrupção). **Antes de cada lote de corridas apaga estes dois ficheiros**, para não misturar com corridas antigas. Cada linha tem um `label` (cenário mais parâmetros) e um `run` (a corrida).

### Cenários da Fase 1 (da proposta)

| # | Cenário | scenario | shockOn | collarSet | nChartists | haltShort | Stop time |
|---|---|---|---|---|---|---|---|
| 1 | Baseline: calibração e validação | baseline | false | 2 | 30 | 3 | 127500 (250 dias) |
| 2 | Choque grande com e sem colares (BES) | shock | true | 2 e 0 | 30 | 3 | 61200 |
| 3 | Choque com muitos chartists (março 2020) | herd | true | 2 e 0 | 60 | 3 | 61200 |
| 4 | Grelha de colares × duração | grid | true | 0, 1, 2, 3 | 30 | 3 e 10 | 61200 |

`collarSet`: 0 = sem colares, 1 = 2%/5%, 2 = 3%/8% (atual), 3 = 5%/10%. O choque (`shockSize` = −10%) acontece no dia 60 (`shockTime` = 30600).

### Criar uma experiência de cenários (Parameters Variation)

1. Projects → botão direito em `Euronext_Main` → **New** → **Experiment** → **Parameters Variation** → Name `Cenario2` → Finish.
2. Propriedades → **Parameters**: para cada parâmetro escolhe **Fixed** (um valor), **Range** (Min, Max, Step) ou **Freeform** (uma lista de valores). Cenário 2: `scenario` = "shock", `shockOn` = true, `collarSet` Freeform 0 e 2.
3. **Replications**: marca *Use replications* e *Fixed number of replications* = **30** (Monte Carlo).
4. **Randomness**: *Fixed seed*. Assim o “com colares” e o “sem colares” usam as mesmas sementes: é o mesmo choque com a mesma sorte, como promete a proposta.
5. **Model time**: Stop at specified time = 61200.
6. Em **Advanced**, desmarca **Allow parallel evaluations** (duas corridas a escrever ao mesmo tempo no mesmo CSV estragam o ficheiro).
7. Run (não há animação, espera que acabe). Os CSV de `model/output/` não vão para o GitHub, porque são grandes. Avisa o João, que corre o `kpis.py` e faz o `kpis.xlsx` (vê a dica abaixo).

### Grelha (cenário 4 e pergunta 1 da proposta)

Parameters Variation com `collarSet` Range 0–3 (step 1), `haltShort` Freeform 3 e 10, e `nChartists` Range 10–60 (step 10). São 4 × 2 × 6 = 48 combinações; com 30 replicações dá 1440 corridas. Começa com 5 replicações para ver quanto tempo demora.

### Otimização (os limiares que minimizam a queda)

1. New → Experiment → **Optimization**. Objective: **minimize** `root.maxDrawdown`.
2. Parameters: `collarSet` Fixed 4; `customDyn` *continuous* 0.01–0.06; `customStat` *continuous* 0.04–0.15; `shockOn` true.
3. Replications: 10 por iteração.
4. **Requirements**: expressão `root.stock.volume`, lower bound = 90% do volume médio do cenário 2 com colares (está no kpis.xlsx). É o "sem perder demasiada liquidez".
5. Number of iterations 200 → Run. A melhor combinação aparece na tabela da experiência.

Para não andarem a passar CSV de um lado para o outro, o mais simples é o João correr o `kpis.py` e o `compare_model.py` no teu PC: ele instala o Python lá uma vez, ou fazem-no juntos numa chamada.

## 4. Calibração com o João (semana 4)

1. Apaga os CSV de `model/output/` e corre o cenário 1 (baseline) com 10 replicações e Stop 127500.
2. O João compara com o BCP real (`compare_model.py`) e diz-te o que não bate.
3. Escolhem juntos o que mudar (a tabela de afinação está no guião dele, secção 5). Tu mudas os parâmetros no AnyLogic e corres outra vez. Muda uma coisa de cada vez.

## 5. Fase 2: LeveragedTrader (semana 5)

Investidor com dinheiro emprestado. Se o preço cai, vende por stop-loss; se a margem fica abaixo do mínimo, recebe uma chamada de margem (margin call) e é obrigado a vender. É isto que cria as cascatas da pergunta 3. Faz na tua cópia e avisa o Miguel para integrar.

1. Parâmetros no Main: `nLeveraged` 10, `leverage` 3, `stopLoss` 0.05, `maintenanceMargin` 0.25.
2. População `leveraged` (type `LeveragedTrader`, Initial number `nLeveraged`). Variáveis: `cash` 5000 (capital próprio), `loan` 0, `position` 0, `entryPrice` 0.
3. Função **margin** (Returns double): `double v = position * main.stock.mid(); return v > 0 ? (v - loan) / v : 1;`
4. Statechart (as regras de como desenhar estão no capítulo 0; os do João são um bom exemplo):

```
Normal --Rate 0.05--> Open (entry: compra alavancado) --T0--> Holding
Holding --Rate 1--> <check> --preço < entry*(1-stopLoss)----> StopLoss   (entry: vende tudo a mid*0.995)
                      |-----margin() < maintenanceMargin---> MarginAlert --Timeout 5--> <again>
                      '-----default-----------------------> Holding        |--margem ainda baixa--> ForcedSale
StopLoss / ForcedSale --T0--> Normal  (ou Liquidated se cash <= 0)           '--default------------> Holding
```

```
// Open -> Entry action
double p = main.stock.mid() * 1.002;
int q = (int) (cash * main.leverage / p);
loan = cash * (main.leverage - 1);
entryPrice = p;
main.brokers.get(0).order(this, 1, p, q);     // as ordens passam pelo broker da Matilde

// MarginAlert -> Entry action (avisa o broker)
send("MARGIN_CALL", main.brokers.get(0));

// ForcedSale -> Entry action (vende tudo, muito abaixo do mercado)
main.brokers.get(0).order(this, -1, main.stock.mid() * 0.98, position);
```

Enquanto o broker da Matilde não estiver pronto, usa `main.stock.order(...)` em vez de `main.brokers.get(0).order(...)`.

## 6. Semana 6

- Corridas finais: cenários com 30 replicações, a grelha e a otimização. Avisa o João para fazer o `kpis.xlsx` final.
- Relatório: como o modelo é instrumentado (que dados grava), os cenários e as experiências (Parameters Variation, replicações, sementes fixas, otimização) e o investidor alavancado.